

Skill week – 5

Session -9

Task – 1

1.To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Test Navigation

```
ganesh@DESKTOP-79N3772:~$ pwd
/home/ganesh
```

`pwd` means **Print Working Directory**.

It shows where you are currently located.

Example:

```
/home/ganesh
```

```
ganesh@DESKTOP-79N3772:~$ mkdir Q1_shell
ganesh@DESKTOP-79N3772:~$ cd Q1_shell
ganesh@DESKTOP-79N3772:~/Q1_shell$ pwd
/home/ganesh/Q1_shell
```

`cd` means **Change Directory**.

This moves you into the `Q1_shell` folder.

```
ganesh@DESKTOP-79N3772:~/Q1_shell$ mkdir dir1
ganesh@DESKTOP-79N3772:~/Q1_shell$ mkdir dir2
ganesh@DESKTOP-79N3772:~/Q1_shell$ ls
dir1  dir2
```

```
ganesh@DESKTOP-79N3772:~/Q1_shell$ nano q1_shell.c
ganesh@DESKTOP-79N3772:~/Q1_shell$ ls
dir1  dir2  q1_shell.c
```

```

GNU nano 8.7.1                                q1_shell.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>
#include <errno.h>

#define SHELL_MAX_INPUT 1024

void show_current_directory()
{
    char current_dir[PATH_MAX];

    if (getcwd(current_dir, sizeof(current_dir)) == NULL)
    {
        perror("getcwd");
        return;
    }

    printf("Current Working Directory: %s\n", current_dir);
}

void change_directory(char *path)
{
    char old_dir[PATH_MAX];

```

```

        if (home == NULL)
        {
            fprintf(stderr, "Error: HOME is not set\n");
            return;
        }

        path = home;
    }

    if (strcmp(path, "-") == 0)
    {
        char *oldpwd = getenv("OLDPWD");

        if (oldpwd == NULL)
        {
            fprintf(stderr, "cd: OLDPWD not set\n");
            return;
        }

        path = oldpwd;
    }

    if (chdir(path) != 0)
    {
        perror("cd");
        return;
    }

    setenv("OLDPWD", old_dir, 1);

```

```

ganesh@DESKTOP-79N3772: ~/Q1_shell
    else if (strncmp(input, "cd ", 3) == 0)
    {
        char *path = input + 3;

        if (strlen(path) == 0)
        {
            change_directory(NULL);
        }
        else
        {
            change_directory(path);
        }
    }
    else if (strcmp(input, "cd") == 0)
    {
        change_directory(NULL);
    }
    else
    {
        printf("Error: Command not found: %s\n", input);
    }
}

return 0;
}
ganesh@DESKTOP-79N3772:~/Q1_shell$ _

```

```

ganesh@DESKTOP-79N3772:~/Q1_shell$ gcc q1_shell.c -o q1_shell
ganesh@DESKTOP-79N3772:~/Q1_shell$ ls -l
total 32
drwxr-xr-x 2 ganesh ganesh 4096 Sep 15 09:04 dir1
drwxr-xr-x 2 ganesh ganesh 4096 Sep 15 09:05 dir2
-rwxr-xr-x 1 ganesh ganesh 16784 Sep 15 09:06 q1_shell
-rw-r--r-- 1 ganesh ganesh 2439 Sep 15 09:06 q1_shell.c
ganesh@DESKTOP-79N3772:~/Q1_shell$ ./q1_shell
=====

```

```

=====
QUESTION 1 - DIRECTORY NAVIGATION
=====
Current Working Directory: /home/ganesh/Q1_shell
q1-shell> pwd
Current Working Directory: /home/ganesh/Q1_shell
q1-shell> cd dir1
Current Working Directory: /home/ganesh/Q1_shell/dir1
q1-shell> pwd
Current Working Directory: /home/ganesh/Q1_shell/dir1
q1-shell> cd ..
Current Working Directory: /home/ganesh/Q1_shell
q1-shell> pwd
Current Working Directory: /home/ganesh/Q1_shell
q1-shell> cd -
Current Working Directory: /home/ganesh/Q1_shell/dir1
q1-shell> Q1_shell/dir2

```

```

q1-shell> hello
Error: Command not found: hello
q1-shell> cd dir1
cd: No such file or directory
q1-shell> pwd
Current Working Directory: /home/ganesh/Q1_shell/dir1
q1-shell> cd ..
Current Working Directory: /home/ganesh/Q1_shell
q1-shell> pwd
Current Working Directory: /home/ganesh/Q1_shell
q1-shell> nano q1_shell.c
Error: Command not found: nano q1_shell.c
q1-shell> ^C

```

Observation:

The mini shell successfully changes the current working directory using the `cd` command. The current directory is displayed using `pwd`. The shell also supports `cd ..` for moving to the parent directory and `cd -` for returning to the previous directory. Invalid paths and invalid commands are detected and appropriate error messages are displayed. Thus, directory navigation, path validation, working-directory maintenance, previous-directory handling, and error handling were successfully implemented and tested.

Task – 2

To Identify Built-ins, Create Dispatch Table, Execute In-Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
ganesh@DESKTOP-79N3772:~$ pwd
/home/ganesh
ganesh@DESKTOP-79N3772:~$ _
```

```
ganesh@DESKTOP-79N3772:~$ mkdir Q2_dispatch
ganesh@DESKTOP-79N3772:~$ _
```

```
ganesh@DESKTOP-79N3772:~$ mkdir Q2_dispatch
ganesh@DESKTOP-79N3772:~$ cd Q2_dispatch
ganesh@DESKTOP-79N3772:~/Q2_dispatch$ pwd
/home/ganesh/Q2_dispatch
ganesh@DESKTOP-79N3772:~/Q2_dispatch$ _
```

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT_SIZE 1024
#define MAX_ARGS 100

/* Function prototypes */
void builtin_pwd(char *args[]);
void builtin_cd(char *args[]);
void builtin_help(char *args[]);
void builtin_exit(char *args[]);

/* Structure for dispatch table */
typedef struct
{
    char *command;
    void (*function)(char *args[]);
} Builtin;

/* Dispatch table */
Builtin builtins[] =
{
    {"pwd", builtin_pwd},
    {"cd", builtin_cd},
    {"help", builtin_help},
    {"exit", builtin_exit}
};

/* Number of built-in commands */
int get_builtin_count()
{
    return sizeof(builtins) / sizeof(builtins[0]);
}

/* pwd built-in */
void builtin_pwd(char *args[])
{
    char current_directory[1024];
```

task2.sh

```

/* cd built-in */
void builtin_cd(char *args[])
{
    char old_directory[1024];

    if (getcwd(old_directory, sizeof(old_directory)) == NULL)
    {
        perror("getcwd");
        return;
    }

    if (args[1] == NULL)
    {
        char *home = getenv("HOME");

        if (home == NULL)
        {
            printf("cd: HOME not set\n");
            return;
        }

        if (chdir(home) != 0)
        {
            perror("cd");
            return;
        }
    }
    else if (strcmp(args[1], "-") == 0)
    {
        char *oldpwd = getenv("OLDPWD");

        if (oldpwd == NULL)
        {
            printf("cd: OLDPWD not set\n");
            return;
        }

        if (chdir(oldpwd) != 0)
        {
            perror("cd");

```

```

ganesh@DESKTOP-79N3772:~/Q1_shell$ ./task2
=====
TASK 2 - MINI SHELL
=====
Built-ins: pwd, cd, exit
Dispatch table initialized.
=====
myshell>

```

```

=====
TASK 2 - MINI SHELL
=====
Built-ins: pwd, cd, exit
Dispatch table initialized.
=====

myshell> pwd
/home/ganesh/Q1_shell
myshell> cd ..
Directory changed successfully.
myshell> pwd
/home/ganesh
myshell> cd -
Directory changed successfully.
myshell> pwd
/home/ganesh/Q1_shell
myshell> hello
Invalid command: hello
Available commands: pwd, cd, exit
myshell>

```

Observation:

The mini shell successfully identifies and executes built-in commands using a dispatch table. The `pwd`, `cd`, and `exit` commands are executed within the shell, the directory state is maintained, and invalid commands are detected with appropriate error messages. Thus, built-in command execution, state maintenance, error handling, and dispatch logic were successfully implemented and tested.

Session – 10

Task – 1

Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

```

ganesh@DESKTOP-79N3772:~$ pwd
/home/ganesh
ganesh@DESKTOP-79N3772:~$ echo "Current Directory: $PWD"
Current Directory: /home/ganesh

```

```

ganesh@DESKTOP-79N3772:~$ OLDPWD="$PWD"
ganesh@DESKTOP-79N3772:~$ cd /tmp
ganesh@DESKTOP-79N3772:/tmp$ pwd
/tmp
ganesh@DESKTOP-79N3772:/tmp$ cd "$OLDPWD"
ganesh@DESKTOP-79N3772:~$ pwd
/home/ganesh

```

Display the current directory

```
ganesh@DESKTOP-79N3772:/tmp$ cd "$OLDPWD"
ganesh@DESKTOP-79N3772:~$ pwd
/home/ganesh
ganesh@DESKTOP-79N3772:~$ touch temp_test.txt
ganesh@DESKTOP-79N3772:~$ ls temp_test.txt
temp_test.txt
ganesh@DESKTOP-79N3772:~$ rm temp_test.txt
ganesh@DESKTOP-79N3772:~$ ls temp_test.txt
ls: cannot access 'temp_test.txt': No such file or directory
ganesh@DESKTOP-79N3772:~$
```

Restore the saved state

Test cleanup

Then remove it and verify clean up

Exit.

```
/home/ganesh
ganesh@DESKTOP-79N3772:~$ touch temp_test.txt
ganesh@DESKTOP-79N3772:~$ ls temp_test.txt
temp_test.txt
ganesh@DESKTOP-79N3772:~$ rm temp_test.txt
ganesh@DESKTOP-79N3772:~$ ls temp_test.txt
ls: cannot access 'temp_test.txt': No such file or directory
ganesh@DESKTOP-79N3772:~$ true
ganesh@DESKTOP-79N3772:~$ echo $?
0
ganesh@DESKTOP-79N3772:~$ false
ganesh@DESKTOP-79N3772:~$ echo $?
1
ganesh@DESKTOP-79N3772:~$ _
```

Task 1 – Observation

The current directory was successfully retrieved and displayed. Exit status, resource cleanup, and saving and restoring the directory state were also tested successfully.

Task 2 — export and Environment Variables

Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality.

```
ganesh@DESKTOP-79N3772:~$ MYNAME="UbuntuStudent"
ganesh@DESKTOP-79N3772:~$ echo "$MYNAME"
UbuntuStudent
ganesh@DESKTOP-79N3772:~$
```

Creating a variable

```
ganesh@DESKTOP-79N3772:~$ echo "$MYNAME"
UbuntuStudent
ganesh@DESKTOP-79N3772:~$ export MYNAME
ganesh@DESKTOP-79N3772:~$ printenv MYNAME
UbuntuStudent
ganesh@DESKTOP-79N3772:~$
```

Export the variable

```
ganesh@DESKTOP-79N3772:~$ export COURSE="OperatingSystems"
ganesh@DESKTOP-79N3772:~$ echo "$COURSE"
OperatingSystems
ganesh@DESKTOP-79N3772:~$
```

Create another variable environment

```
ganesh@DESKTOP-79N3772:~$ printenv MYNAME
UbuntuStudent
ganesh@DESKTOP-79N3772:~$ printenv COURSE
OperatingSystems
ganesh@DESKTOP-79N3772:~$
```

Check both variables

```
ganesh@DESKTOP-79N3772:~$ export COURSE="LinuxShell"
ganesh@DESKTOP-79N3772:~$ echo "$COURSE"
LinuxShell
ganesh@DESKTOP-79N3772:~$
```

Update an existing variable

```
ganesh@DESKTOP-79N3772:~$ export COURSE=LinuxShell
ganesh@DESKTOP-79N3772:~$ echo "$COURSE"
LinuxShell
ganesh@DESKTOP-79N3772:~$ bash -c 'echo "Child process: $MYNAME"'
Child process: UbuntuStudent
ganesh@DESKTOP-79N3772:~$ bash -c 'echo "Child process course: $COURSE"'
Child process course: LinuxShell
ganesh@DESKTOP-79N3772:~$
```

Test export to a child process

```
ganesh@DESKTOP-79N3772:~$ export 123ABC=hello
-bash: export: `123ABC=hello': not a valid identifier
ganesh@DESKTOP-79N3772:~$
```

Testing an invalid variable name

Task 2 – Observation

Environment variables were successfully created, updated, and exported. Variable-name validation and passing exported variables to child processes were also tested successfully.